Proyecto Integrador Fase 1

Odalis Anangonó, Paula Cevallos y Keyla Imba

Facultad de Ingenierías Digitales y Tecnologías Emergentes, Universidad Internacional del Ecuador

Programación Orientada a Objetos

Mgt. María del Carmen Salazar Torres

8 de octubre de 2026

1.- El problema y la necesidad de que le surge a la empresa

La empresa desarrolla empezó a implementar soluciones tecnológicas y acaba de abrir una segunda sede y lo que se necesita es lo siguiente:

- Que la empresa mantenga comunicación con la segunda sede.
- Ofrecer los mismos servicios entre la empresa y la sede
- Aplicar autenticadores para que los que realmente son usuarios puedan entrar.
- Proteger la información.

Lo que se realizará es una aplicación enfocada en una empresa en Go que cubra las necesidades y funciones sobre la red de la empresa y su servicio de autenticación.

1.1.- Sistema elegido

Es un sistema de mensajería que permite a los usuarios de la empresa y de la sede comunicarse entre sí de forma sencilla. Cada usuario cuenta con una bandeja de entrada desde la que puede enviar, recibir, leer y eliminar mensajes. Todo funciona a través de nuestro propio servidor, sin necesidad de utilizar servicios de correo externos como SMTP.

1.2.- Por qué se escogió este problema

Problema que resuelve: Escogimos este proyecto porque buscamos una forma sencilla y organizada de mejorar la comunicación entre los usuarios de la empresa y la sede, sin depender de servicios de correo externos.

Operaciones cliente-servidor reales: Lo elegimos porque nos permite poner en práctica cómo se comunican un cliente y un servidor mediante acciones como enviar, leer y eliminar mensajes.

Datos medibles para Estadística: Este proyecto nos permite obtener datos reales sobre el uso del sistema, como la cantidad de mensajes enviados, recibidos, leídos y eliminados, que después podemos analizar estadísticamente.

Encaja con POO: Lo escogimos porque podemos aplicar la programación orientada a objetos creando elementos como usuarios, mensajes, bandejas de entrada y el servidor, lo que facilita organizar y desarrollar el sistema.

1.3.- Datos para Estadística

- Número de correos que se enviaron.
- Tiempo que tarda el servidor en responder cada operación en milisegundos.
- Porcentajes de errores en operaciones.
- Proporción de correos leídos y no leídos.
- Intentos de inicio de sesión logrados y fallidos.

Estos datos se almacenan como parte del registro de actividad del sistema y permiten llevar un control de los mensajes enviados, recibidos y eliminados.

1.4.- Actores y usuarios iniciales

- Empleado de la matriz: puede enviar, recibir, leer y eliminar correos desde su ubicación.
- Empleado de la sede: cuenta con las mismas funciones para comunicarse con los usuarios de la matriz.
- Administrador del sistema: se encarga de crear usuarios y sus bandejas de entrada, además de revisar las estadísticas de uso.
- Servidor de correo (Go): recibe las solicitudes de los usuarios, almacena los mensajes y se encarga de entregarlos a sus destinatarios.
- Servicio de autenticación (FastAPI): verifica el acceso de los usuarios y genera una sesión válida para que puedan ingresar a su bandeja de entrada y enviar mensajes de forma segura.

1.5.- Requisitos mínimos

Tabla 1

Requisitos funcionales y no funcionales del sistema de correo interno

Código	Requisito
RF-SW-01	Un usuario de una sede usa una funcionalidad del servidor y recibe una respuesta válida.
RF-SW-02	Enviar, recibir, leer y eliminar correos se completa sin errores críticos y la información persiste.
RF-SW-03	Acceder a la bandeja o enviar mensajes exige una sesión válida del servicio de autenticación.
RNF-04	Código modular, configuraciones documentadas y repositorio Git organizado.
RNF-06	Se pueden agregar usuarios o buzones nuevos sin rediseñar todo el sistema.

Nota: Elaboración propia 9 de septiembre del 2026. Estos requisitos describen lo que debe hacer el sistema de correo interno y las características que necesita para funcionar correctamente. Elaboración propia 9 de Septiembre del 2026

1.6.- Cronograma de trabajo del equipo

Tabla 2

Cronograma de planificación y el desarrollo del proyecto

Fase	Semanas	Qué se hace	Entregable
1. Contexto, planificación y organización	1–2	Definir el problema, elegir el sistema, asignar roles y crear el repositorio en GitHub.	Documentos de la Fase 1 y repositorio
2. Análisis y requisitos	3–5	Requisitos funcionales y no funcionales con código, acciones de usuario medibles y revisión de riesgos.	Requisitos y matriz de riesgos
3. Diseño de la solución	6–7	Estructura del software, la red y la seguridad; VLSM y VLAN; boceto UI/UX.	Diseño de software y red
4. Prototipo y desarrollo inicial	8–10	Módulos en Go, red inicial, autenticación y prueba piloto de recolección de datos.	Prototipo funcional
5. Integración y despliegue	11–12	Conectar Go con FastAPI y desplegar la red (OSPF, NAT, DHCP, etc.).	Sistema integrado
6. Pruebas y validación	13–14	Pruebas funcionales y de seguridad, análisis estadístico e impacto socio tecnológico.	Informe de pruebas
7. Presentación final	15	Demostración en vivo de las funciones y del impacto.	Presentación
8. Entrega definitiva	16	Revisión final y verificación de datos.	Repositorio final

Nota: Elaboración propia 9 de septiembre del 2026. El cronograma muestra cómo se organizará el proyecto a lo largo de las 16 semanas, desde la planificación hasta la entrega del sistema de correo interno. En cada fase se detallan las actividades que se realizarán y los resultados que se esperan obtener.

2.- Actores, roles y responsabilidades

El sistema cuenta con cinco actores y el equipo de trabajo se organiza en diferentes áreas para desarrollar el proyecto. Estas incluyen el desarrollo de software en Go, la configuración de redes, la seguridad, el análisis de datos y el análisis del impacto socio tecnológico. Cada integrante tiene una responsabilidad principal, pero todas colaboran entre sí para cumplir los objetivos del proyecto.

Tabla 3

Actores y funciones principales del sistema de correo interno

Actor	Qué hace en el sistema
Empleado de la empresa	Inicia sesión, envía, recibe, lee y elimina correos.
Empleado de la sede	Hace lo mismo pero desde la segunda sede.
Administrador	Crea usuarios y buzones; revisa métricas y errores.
Servidor de correo (Go)	Procesa peticiones, guarda y entrega los mensajes.
Servicio de autenticación (FastAPI)	Verifica credenciales y emite la sesión válida.

Nota: Elaboración propia, 9 de octubre del 2026. Se describen los actores que participan en el sistema y las funciones que cumple cada uno para permitir el envío, la recepción y la administración de los correos de forma organizada y segura.

2.1.- Roles del equipo

Tabla 4

Roles y responsabilidades de las integrantes del equipo

Integrante	Rol	Responsabilidades	Apoya en
Paula Doménica Cevallos Vacacela	Desarrolladora Lead Go / POO y Coordinación Técnica	Diseño de clases y structs, servidor y cliente en Go, manejo de errores, UML.	Documentación y repositorio
Odalis Jamel Anangonó Sevillano	Especialista en Infraestructura, Redes y Seguridad	Direccionamiento VLSM, VLAN, enrutamiento (OSPF), NAT y DHCP.	Pruebas de conexión cliente-servidor
Keyla Antonella Imba Untuña	Analista Estadístico y Socio-Tecnológico	Integración con FastAPI, sesiones, protección de datos, análisis de riesgos.	Datos y análisis estadístico

Nota: Elaboración propia, 9 de octubre del 2026. Se detallan las funciones asignadas a cada integrante, sus principales tareas y las áreas en las que colaborará durante el desarrollo del proyecto.

3. Contrato de comunicación del equipo

El presente contrato de comunicación establece acuerdos, normas de interacción, tiempos de respuesta, procedimientos para la toma de decisiones y procesos para la resolución de conflictos que regirán las actividades del equipo de trabajo durante el desarrollo del proyecto integrador.

3.1 Canales oficiales de comunicación

Para asegurar un flujo de información eficiente y contínua, el equipo ha decidido utilizar los siguientes medios de comunicación:

- WhatsApp como canal principal e inmediato. En este canal se realizará la coordinación diaria, alertas, recordatorios de entregas y notificaciones de alta importancia.
- Instagram como canal complementario e informal, que servirá como medio para compartir enlaces de referencia, recursos visuales breves y plan de contingencia en caso de que el canal principal se encuentre inhabilitado para todos o alguno de los miembros del equipo.
- Jitsi Meet para sesiones de trabajo en vivo con el objetivo de lograr el diseño colaborativo de la arquitectura del software en GO, la coordinación de la infraestructura de red, y la revisión de avances semanales.

- Finalmente, para el gestión de código y tareas se ha decidido usar GitHub, ya que aquí se puede realizar el registro formal de código necesario, la gestión de problemas, pull requests y seguimiento de tareas pendientes.

3.2 Tiempo de respuesta

Con el punto de mantener un ritmo de trabajo contínuo sin comprometer la individualidad de cada integrante, se han fijado los siguientes lapsos de respuesta:

Días hábiles (de lunes a viernes):

- Máximo 2 horas como tiempo de respuesta a mensajes y consultas urgentes por el medio de comunicación principal.
- Máximo 8 horas como tiempo de respuesta a consultas y archivos generales en los canales digitales secundarios.

Esto tendrá una excepción durante las 48 horas previas al plazo límite de entrega del proyecto en su totalidad, reduciendo el tiempo de respuesta por el canal principal a 1 hora obligatoria.

3.3 Mecanismos para la Toma de decisiones

Las decisiones de carácter técnico, organizativo y metodológico se realizarán tomando en cuenta las siguientes reglas:

- **Principio de mayoría simple:** Las decisiones que tienen que ver con la distribución de tareas, ajuste de formatos y cronogramas de reuniones entrarán a una votación, siendo aprobadas por mayoría simple de los integrantes.
- **Criterio de Roles técnicos**: En caso de que se llegue a un empate técnico con referencia a la implementación de software en GO, la arquitectura de red o bien la seguridad, prevalecerá la recomendación realizada por el integrante designado como líder de la asignatura respectiva.
- **Aprobación por silencio administrativo:** Si cualquiera de los tres integrantes no emite su decisión en un lapso de 24 horas tras la publicación de consultas formales en el grupo de WhatsApp, se entenderá su aceptación de la postura decidida por la mayoría del equipo.

3.4 Procedimiento de escalación y resolución de conflictos

Para llegar a la resolución de cualquier discrepancia interpersonal, incumplimiento de tareas, o retrasos no justificados, el equipo ha decidido aplicar un modelo de escalación en tres niveles:

- Nivel 1: Diálogo directo: Ante el primer 'strike' o desacuerdo entre cualquiera de los integrantes, las partes involucradas han de sostener un diálogo directo y privado dentro de las primeras 24 horas para revisar posibles causas del problema y soluciones a la par.
- Nivel 2: Reunión de contingencia: Si los problemas persisten pasadas las 48 horas sin una explicación razonable, el equipo convocará una reunión extraordinaria mediante Jitsi Meet para reevaluar los papeles de responsabilidad.

- Nivel 3: Escalación académica: Si un miembro decide incumplir de forma reiterada con sus respectivas responsabilidades, afectando consecuentemente el avance grupal, el equipo recopilará evidencias registradas y solicitará la mediación de los docentes tutores del proyecto integrador.

4. Repositorio Git inicial con README

4.1 Repositorio

Para asegurar la trazabilidad del código fuente, el control de versiones y respaldo de la documentación, se ha asignado un repositorio público en la plataforma de GitHub (https://github.com/TownECreator/correos-integrador)

4.2 Selección y justificación de la licencia

En conjunto, se ha seleccionado la licencia MIT para la distribución del proyecto, basándonos en la naturaleza permisiva y flexible de esta licencia, lo cuál facilita el uso académico, la revisión por parte de los docentes y la eventual reutilización de los componentes del proyecto sin restricciones grandes. Del mismo modo, la licencia MIT asegura la protección legal del equipo mediante una cláusula clara de extensión de responsabilidad ante fallos de funcionamiento en entornos de producción externos.

5. Diagrama de clases UML inicial (gestión de correos internos)

